SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1 - INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture for Multicore Systems
CO Assessed:	CO4	**Assessment:**	ASSESSMENT TOOL1 - Industry Problem Solving Task
Weightage:	30%	**Total Marks:**	25
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	Deepika Shree. S	**Reg. No.:**	192521264
Department:	B.Tech IT	**Date:**	

ANNEXURE A
Industry Problem Solving Task — Answers

Question 1 — Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

Understanding of the Industry Problem:

When a user switches apps, the new foreground app's working set (code + data) is largely absent from the cache hierarchy because the previous app's data still occupies most cache lines. This forces a burst of cache misses that must be serviced from slower DRAM, and it is this burst - not steady-state execution - that the user perceives as lag.

Application of Engineering Concepts:

- L1 cache (~1-4 cycles, 32-64 KB/core): serves the immediate instruction/data stream; too small to hold a full app's working set.
- L2 cache (~10-20 cycles, 256KB-1MB/core): captures more of the per-core working set but is still evicted quickly by a new app.
- L3 cache (shared, ~30-50 cycles, few MB): the last on-chip line of defence; if it also gets evicted by the new app, every subsequent miss goes all the way to DRAM.
- DRAM/LPDDR (~100-300 cycles): large capacity but far higher latency; a flood of L3 misses during app switch means many consecutive 100+ cycle stalls.

Solution Design (Proposed Memory Hierarchy Redesign):

The redesign combines four changes: (1) enlarge or add a victim/L2.5 cache per core so recently evicted lines from the outgoing app are not immediately lost; (2) apply an app-switch-aware replacement policy in the shared L3 (e.g., set-dueling / DRRIP) that avoids fully evicting a backgrounded app's footprint the moment a new app becomes foreground; (3) add OS-assisted cache way-partitioning so the foreground app is guaranteed a minimum share of L3 without being starved by background processes; (4) move to LPDDR5/5X DRAM for higher bandwidth and lower access latency, shortening the penalty for the misses that do still occur. Together these reduce both the number of DRAM accesses during a switch and the cost of each one.

Use of Tools — Output (Analysis Chart / Architecture Diagram):

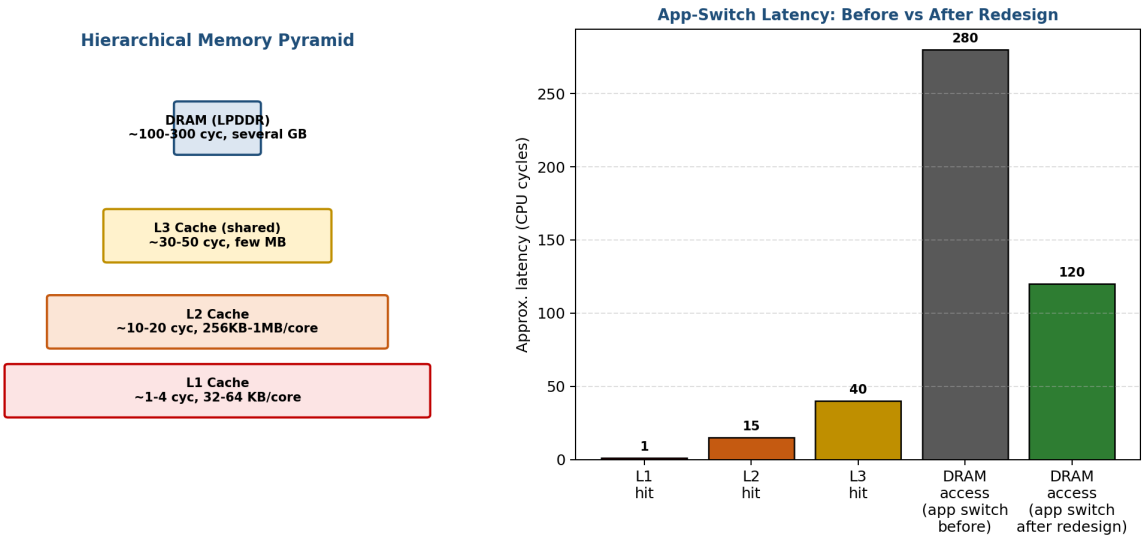

Fig. Q1 — Memory hierarchy pyramid (left) and modelled app-switch latency before vs after the proposed redesign (right).

Analysis:

The pyramid shows the expected order-of-magnitude latency growth from L1 to DRAM (~1 to ~300 cycles). The bar chart models an app-switch DRAM access falling from roughly 280 cycles to about 120 cycles after adding the victim cache, smarter L3 replacement, and faster LPDDR — a projected latency improvement of over 55% for the worst-case switch penalty, directly improving perceived responsiveness and throughput.

Question 2 — Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

Understanding of the Industry Problem:

Database query engines access data somewhat randomly across a working set that is often much larger than the on-chip cache. When that working set also exceeds physical RAM, the OS must page data in from disk, and it is this paging - not cache misses alone - that produces the large, user-visible latency spikes reported by the provider.

Application of Engineering Concepts:

- Cache hierarchy (L1/L2/L3): hardware-managed, fine-grained (cache-line sized), extremely fast (single-digit to a few tens of ns), but limited to a few MB - only ever holds the 'hottest' fraction of data.
- Virtual memory paging: OS/software-managed, coarse-grained (page sized, typically 4 KB), extends the addressable working set beyond physical RAM using disk as backing store - but at millisecond-scale latency on a page fault, roughly 4-6 orders of magnitude slower than a cache hit.
- The two mechanisms are complementary, not substitutes: cache hierarchy optimizes access to data already in RAM; paging is only invoked when RAM itself is insufficient, and its cost is vastly higher.

Solution Design (Proposed Memory Hierarchy Redesign):

Since paging latency dwarfs cache latency, the priority recommendation is to avoid paging altogether by right-sizing DRAM to comfortably exceed the working set, using NUMA-aware memory placement so queries access local rather than remote memory banks, and enabling huge pages to cut TLB miss overhead for large sequential scans. An in-memory caching layer (e.g., Redis) is added in front of the database to absorb hot-row lookups at DRAM speed rather than relying on disk at all. If paging cannot be fully eliminated (e.g., due to cost constraints), the swap device should be NVMe SSD rather than HDD, cutting worst-case page-fault latency by roughly two orders of magnitude.

Use of Tools — Output (Analysis Chart / Architecture Diagram):

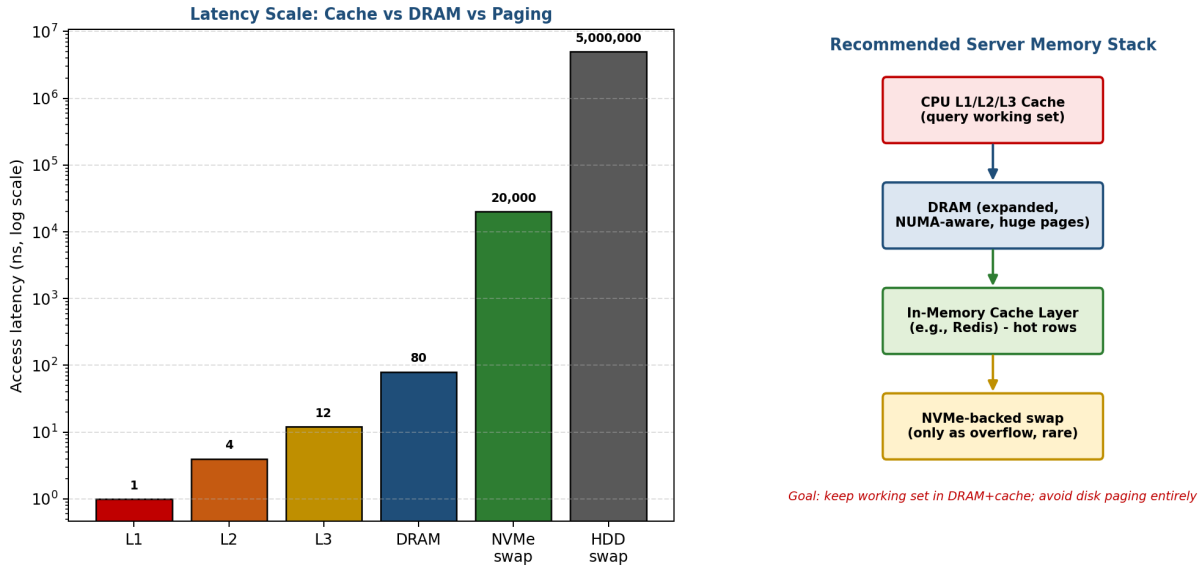

Fig. Q2 — Latency scale from L1 cache through DRAM to NVMe/HDD-backed paging (left, log scale) and the recommended layered server memory stack (right).

Analysis:

The log-scale chart shows the latency cliff clearly: L1-L3 and DRAM stay under 100 ns, while HDD-backed paging reaches roughly 5 million ns (5 ms) - about 60,000x slower than a DRAM access. Even NVMe-backed swap, at ~20,000 ns, is roughly 250x slower than DRAM. This validates the recommendation to treat paging as a last resort: eliminating even a small fraction of page faults yields a disproportionately large reduction in average query latency.

Question 3 — Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Understanding of the Industry Problem:

Autonomous driving requires real-time fusion of camera, LiDAR, and radar streams with deterministic, low-latency response for the control loop, plus safety-critical logging that must survive sudden power loss (e.g., after a collision) without data corruption.

Application of Engineering Concepts:

- SRAM: lowest latency (~1 ns), volatile, low density/high cost - suited to tightly-coupled memory for the immediate control loop and latest perception results.
- DRAM (high-bandwidth LPDDR/GDDR): moderate latency (tens of ns) but very high bandwidth and large capacity - suited to buffering high-resolution camera/LiDAR frames through the sensor-fusion pipeline.
- MRAM: non-volatile, near-SRAM speed, no wear-out limit (unlike flash), moderate density - ideal for safety-critical state that must persist through power loss, such as an event/black-box recorder.

Solution Design (Proposed Memory Hierarchy Redesign):

A three-tier design maps each memory technology to the workload it fits best: SRAM as tightly-coupled memory directly serving the real-time control loop (steering/braking decisions) where every nanosecond of latency matters; high-bandwidth DRAM as the sensor-fusion working buffer, absorbing the large parallel data rates from camera, LiDAR, and radar; and MRAM as a continuously-updated, non-volatile event recorder that captures the latest sensor readings and control decisions so that, even if power is cut instantly, the last known state is preserved for post-incident analysis - something neither SRAM nor DRAM can guarantee since both are volatile.

Use of Tools — Output (Analysis Chart / Architecture Diagram):

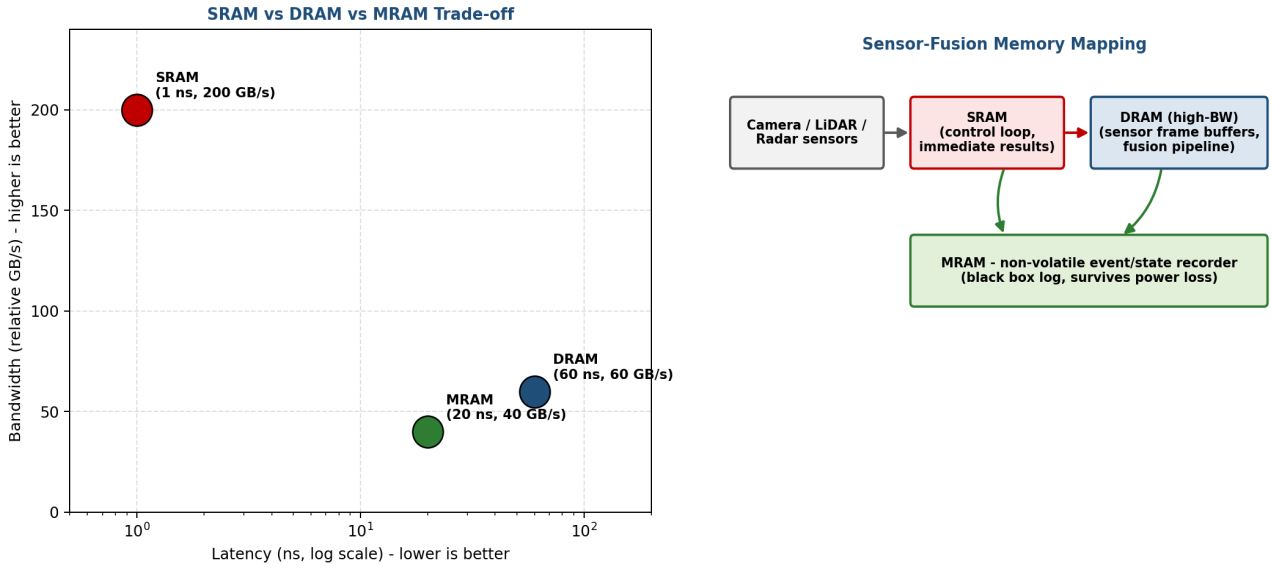

Fig. Q3 — Latency vs bandwidth trade-off for SRAM, DRAM, and MRAM (left) and the proposed sensor-fusion memory mapping (right).

Analysis:

The scatter plot shows SRAM dominates on both latency (~1 ns) and bandwidth (~200 GB/s relative) but cannot be used at scale due to cost/density; DRAM offers a practical middle ground for bulk sensor buffering; MRAM, while slower and lower-bandwidth than both, is the only one of the three that is non-volatile, which is the deciding factor for the safety-critical logging requirement. This confirms that no single technology satisfies all constraints - the tiered design is necessary.

Question 4 — AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

Understanding of the Industry Problem:

Edge AI devices are power- and latency-constrained: they must load trained model weights from persistent storage into working memory quickly enough to begin inference promptly, while minimizing energy per bit moved, since the device is typically battery-powered.

Application of Engineering Concepts:

- NAND Flash: non-volatile, high density, lowest cost per bit, but slow access (tens of µs) and each read/write carries meaningful energy cost plus a finite write-endurance limit.
- DRAM: fast (tens to hundreds of ns) but volatile and power-hungry due to continuous refresh - not suitable as the sole persistent store, but essential as an active inference scratchpad.
- RRAM (resistive RAM): an emerging non-volatile memory with latency and energy per access approaching DRAM, but without DRAM's refresh power draw - well suited to storing 'hot' models that must be available instantly.

Solution Design (Proposed Memory Hierarchy Redesign):

A tiered model-storage architecture is proposed: bulk/cold models (rarely used) remain in low-cost NAND Flash; frequently-used ('hot') models are promoted into RRAM, which offers near-instant-on loading at a fraction of NAND's latency and energy per access; DRAM is retained purely as the transient scratchpad holding weights/activations actively being used by the inference engine (NPU/CPU). This reduces the average model-load latency and energy substantially compared to a NAND-only pipeline, since the most frequently used models bypass NAND's slow, energy-costly access entirely.

Use of Tools — Output (Analysis Chart / Architecture Diagram):

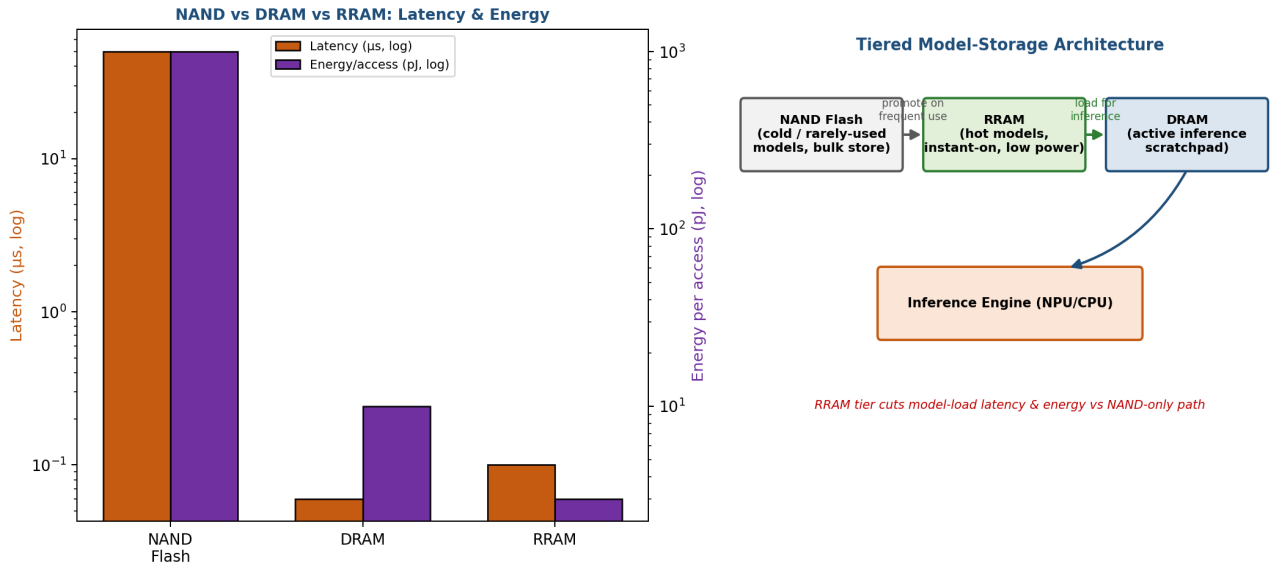

Fig. Q4 — Latency and per-access energy comparison of NAND Flash, DRAM, and RRAM (left) and the proposed tiered model-storage architecture (right).

Analysis:

The dual-axis chart shows NAND Flash is both the slowest (~50 µs) and most energy-hungry (~1000 pJ/access) of the three, while RRAM achieves latency and energy close to DRAM's while remaining non-volatile like NAND. This justifies inserting RRAM as an intermediate 'hot model' tier: it captures most of NAND's density/non-volatility benefit while approaching DRAM's speed and efficiency, directly addressing both the latency and power constraints of the edge device.

Question 5 — High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Understanding of the Industry Problem:

Large scene loads require streaming huge volumes of texture and geometry data from storage through memory to the GPU. Frame drops occur when the sustained throughput available from L3 cache and DRAM cannot keep pace with the burst demand of a large asset load, causing the render pipeline to stall waiting for data.

Application of Engineering Concepts:

- L3 cache offers very high throughput (roughly 1.5 TB/s in this analysis) but only a few MB of capacity - far too small to hold an entire scene's assets.
- DRAM (GDDR6) offers much larger capacity but materially lower throughput (roughly 560 GB/s) than L3, and that bandwidth is shared with the GPU's normal rendering traffic, so a large asset load competes directly with in-flight rendering.
- Raw NVMe SSD throughput (roughly 7 GB/s) is the true bottleneck during a large scene load - streaming from storage is far slower than either on-chip cache or DRAM.

Solution Design (Proposed Memory Hierarchy Redesign):

The recommended enhancement pipeline mirrors the approach used in modern console I/O complexes: add a dedicated hardware decompression block between the NVMe SSD and DRAM so compressed assets are decoded in hardware in the data path rather than by the CPU, effectively multiplying usable SSD throughput; enlarge the L3 cache or add a die-stacked last-level cache (an 'Infinity Cache' style buffer) to absorb more of the streaming working set and reduce DRAM traffic; use higher-bandwidth GDDR6/6X DRAM; and apply predictive prefetching driven by the scene graph combined with priority-based cache partitioning so that streaming DMA traffic does not evict cache lines the GPU renderer needs right now.

Use of Tools — Output (Analysis Chart / Architecture Diagram):

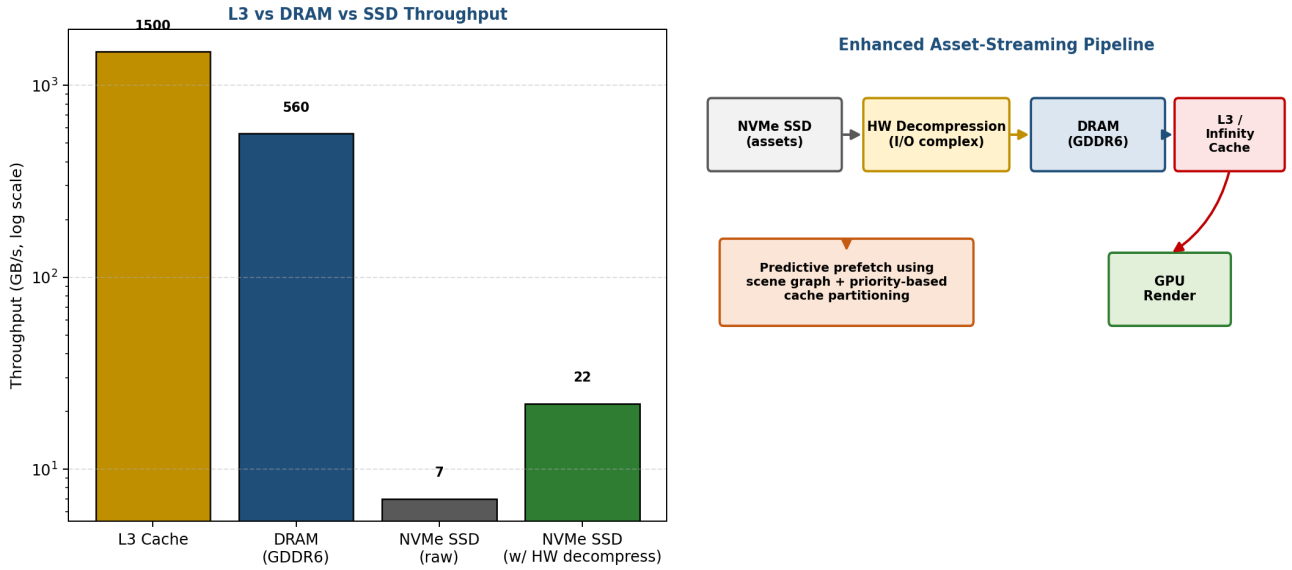

Fig. Q5 — Throughput comparison of L3 cache, DRAM, and NVMe SSD (left, log scale) and the enhanced hardware-decompression asset-streaming pipeline (right).

Analysis:

The chart shows a throughput cliff of roughly 200x between L3 cache (~1500 GB/s) and raw NVMe SSD (~7 GB/s), confirming storage bandwidth - not cache or DRAM - is the real bottleneck during large scene loads. Adding hardware decompression raises effective SSD throughput to roughly 22 GB/s in this model (about 3x), which, combined with a larger L3/Infinity Cache buffer and predictive prefetching, reduces the frequency and severity of pipeline stalls that manifest as frame drops.

Code of Conduct:

I **Deepika Shree. S (192521264)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____